Show title & tags

Document number	P2821R4
Date	2023-7-26
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Library Evolution Working Group (LEWG) SG23 Safety and Security

span.at (http://span.at)()

Table of contents

- span.at()
  - Changelog
  - Abstract
  - Request
  - Motivation
    - Safety
    - Consistency
    - Public Relations
  - Wording
  - Feature test macro
  - Implementation Experience
  - Summary
  - References

Changelog

R4

- Added a wording section

R3

- Fixed some typos

R2

- Corrected `Feature test macro`
- Added references to `Usability Enhancements for std::span` [1] [2]

R1

- Added verbiage stating that this is not freestanding due to its throwing an exception
- Added feature test macro section

Abstract

This paper proposes the standard adds the `at` method to `std::span` class in order to address safety, consistency and PR (public relations) concerns.

Request

```
// no freestanding
#if __STDC_HOSTED__ == 1
constexpr reference at(size_type idx) const;
#endif
```

Returns a reference to the element at specified location idx, with bounds checking.

If idx is not within the range of the container, an exception of type std::out_of_range is thrown.

Parameters idx - index of the element to return

Return value Reference to the requested element.

Exceptions Throws std::out_of_range if idx >= size().

Complexity Constant.

Motivation

Safety

This new method is safe in the sense that it has defined behavior instead of undefined behavior. Further, the defined behavior is one that can be caught in the code by catching the exception.

Consistency

The `std::string` [3], `std::string_view` [4], `std::deque` [5], `std::vector` [6] and `std::array` [7], all have both the unsafe `operator[]` and the safe `at` function.

	`operator[]`	`at()`
`std::string`	✓	✓
`std::string_view`	✓	✓
`std::deque`	✓	✓
`std::vector`	✓	✓
`std::array`	✓	✓
`std::span`	✓	✗

Public Relations

`C++` keeps giving easy wins to our rivals even though they like to hit below the belt. It is nice when a supporter of a rival language makes an honest comparison.

“In both Rust and C++, there is a method for checked array indexing, and a method for unchecked array indexing. The languages actually agree on this issue. They only disagree about which version gets to be spelled with brackets.” - **Being Fair about Memory Safety and Performance** [8]

Unfortunately this isn’t totally true.

“ `std::span<T>` ***indexing***” [9]

…

“ `std::vector` and `std::array` can at least theoretically be used safely because they offer an `at()` method which is bounds checked (in practice I’ve never seen this done, but you could imagine a project adopting a static analysis tool which simply banned calls to `std::vector<T>::operator[]`). `span` does not offer an `at()` method, or any other method which performs a bounds checked lookup.” [9:1]

“Interestingly, both Firefox and Chromium’s backports of std::span do perform bounds checks in operator[], and thus they’ll never be able to safely migrate to std::span.” - **Modern C++ Won’t Save Us** [9:2]

Consequently, the programming community in general are encouraged to ask some tough questions.

1. Why was `span::at()` not provided in `C++20` ? [1:1]
2. Was this an accidental omission or was it deliberate? [2:1]
3. If deliberate, what was the rationale?
4. If deliberate, are the other `at` functions going to be deprecated?
5. Why was `span::at()` not added in `C++23` ?
6. Is it going to be in `C++26` ?

Ultimately, this becomes a stereotypical example of how `C++` traditionally handles safety. This example gets to be pointed at for years/decades to come. All of this could have been avoided, along with more effort of adding this function individually had more consideration been given valid safety concerns.

Wording

The wording is relative to N4950 [10].

> Insert the following into [span.overview] (https://eel.is/c++draft/span.overview), header `<span>` synopsis within the span class, immediately after the subscript operator in the element access section:

```
// no freestanding
#if __STDC_HOSTED__ == 1
constexpr reference at(size_type idx) const;
#endif
```

> Insert the following into [span.elem] (https://eel.is/c++draft/span.elem), span's `Element access` section, immediately after the subscript operator:

> constexpr reference at(size_type idx) const;
>
> *Returns*: *(data() + idx).
>
> *Throws*: out_of_range if idx >= size().

Feature test macro

> Insert the following to [version.syn] (https://eel.is/c++draft/version.syn), header `<version>` synopsis:

```
#define __cpp_lib_span_at 20XXXXL // also in <span>
```

Implementation Experience

Both of the `span lite` and `Guidelines Support Library` libraries have this new method implemented for years.

- `span lite: A single-file header-only version of a C++20-like span for C++98, C++11 and later` [11]
- `Guidelines Support Library` [12]

Likely, there are others too such as *"Firefox and Chromium's backports of std::span"*. [9:3]

Summary

Please add the `at` method to `std::span` class in order to address safety, consistency and PR (public relations) concerns. Also keep ones mind open to an `std::expected` version in the future as well as other reasonable safety requests.

References

1. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1024r0.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1024r0.pdf) ↩ ↩

2. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1024r1.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1024r1.pdf) ↩ ↩

3. https://en.cppreference.com/w/cpp/string/basic_string/at (https://en.cppreference.com/w/cpp/string/basic_string/at) ↩

4. https://en.cppreference.com/w/cpp/string/basic_string_view/at (https://en.cppreference.com/w/cpp/string/basic_string_view/at) ↩

5. https://en.cppreference.com/w/cpp/container/deque/at (https://en.cppreference.com/w/cpp/container/deque/at) ↩

6. https://en.cppreference.com/w/cpp/container/vector/at (https://en.cppreference.com/w/cpp/container/vector/at) ↩

7. https://en.cppreference.com/w/cpp/container/array/at (https://en.cppreference.com/w/cpp/container/array/at) ↩

8. https://www.thecodedmessage.com/posts/unsafe/ (https://www.thecodedmessage.com/posts/unsafe/) ↩

9. https://alexgaynor.net/2019/apr/21/modern-c++-wont-save-us/ (https://alexgaynor.net/2019/apr/21/modern-c++-wont-save-us/) ↩ ↩ ↩ ↩

10. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf) ↩

11. https://github.com/martinmoene/span-lite#at (https://github.com/martinmoene/span-lite#at) ↩

12. https://github.com/microsoft/GSL/blob/main/include/gsl/span_ext#L141 (https://github.com/microsoft/GSL/blob/main/include/gsl/span_ext#L141) ↩